CS 473 (Spring 2025) Yiming Qin (yimingq3@illinois.edu)

Problem 6.1

Solution:

(a) Solve by brute force:

Algorithm 1

```
Initialize C* as a circle with infinity radius.
for each triplet of points (p_i, p_j, p_k) and pair of points (p_i, p_j) do
    Construct the unique circle C that passes through all three or two points.
    Count the number of points inside or on the boundary of C.
    if at least n − k points are enclosed and the radius is smaller than C* then
        C* = C.
return C*
```

Complexity Analysis:

There are $\binom{n}{3} + \binom{n}{2} = O(n^3)$ triplets and pairs, so the number of candidate circles is at most $O(n^3)$. Checking each circle takes $O(n)$ time to count the enclosed points. Hence, the total runtime is $O(n^3) \times O(n) = O(n^4)$.

(b)

$$B^* \subseteq R \subseteq S - Q^* \;\Leftrightarrow\; B^* \subseteq R \text{ and } Q^* \cap R = \emptyset$$

Define two independent events:

Event A: All points in B^* are included in R.

Event B: No points from Q^* are included in R.

Since each point is independently selected with probability $1/k$:

$$\Pr(A) = (1/k)^3 = 1/k^3.$$

Similarly, the probability that none of the at most k outlier points are selected is:

$$\Pr(B) = (1 - 1/k)^k.$$

Using the standard bound $(1 - 1/k)^k < 1/e$, $\Pr(B)$ remains a constant $\Theta(1)$.

Thus, the desired probability is:

$$\Pr(B^* \subseteq R \text{ and } Q^* \cap R = \emptyset) = \Pr(A) \cdot \Pr(B) = \frac{1}{k^3} \cdot \Theta(1) = \Omega(1/k^3).$$

(When $k = 1$, we can sample each p with probability $1/2$ so the probability still satisfies $\Omega(1/k^3)$)

(c) From part (b):

$$\Pr(B^* \subseteq R \text{ and } Q^* \cap R = \emptyset) = \Omega(1/k^3).$$

When this event occurs, the subset R contains all necessary boundary points of C^* and excludes all outliers. Consequently, the smallest enclosing circle computed from R must be C^* itself, which covers at least $n-k$ points. Thus, the algorithm produces the correct result with probability at least $\Omega(1/k^3)$.

Algorithm 2

1: Construct a random subset $R \subseteq S$ by including each point $p \in S$ independently with probability $1/k$.
2: Use Welzl's algorithm to compute the smallest enclosing circle C_R for the subset R.
3: **return** C_R

Complexity Analysis:

Iterating over S and independently deciding whether to include each point in R takes $O(n)$ time. Given that the expected size of R is $O(n/k)$, the smallest enclosing circle runs in $O(n)$ worst-case time.

(d) Use the Monte Carlo algorithm from part (c) as a subroutine that, in one iteration, randomly construct R and compute C_R. Scan all points in S to count how many lie inside or on the boundary of C_R. If at least $n-k$ points are contained, output C_R as the solution; otherwise, declare this iteration a failure.

Let p be the probability that one iteration returns the correct circle. From part (b),

$$p = \Pr(B^* \subseteq R \text{ and } R \subseteq S - Q^*) = \Omega(1/k^3).$$

In order to reduce the error probability to at most 0.01, Repeat the above procedure T times independently so that:

$$(1-p)^T \leq 0.01,$$

we obtain,

$$T \geq \left\lceil \frac{\ln 0.01}{\ln(1-p)} \right\rceil.$$

From part (b), $p < 1/e$, thus $\ln(1-p) > -2p$, it suffices to take

$$T = O\left(\frac{1}{p}\right) = O\left(k^3\right).$$

Complexity Analysis:

Each iteration runs in $O(n)$ worst-case time. Repeating the procedure $T = O(k^3)$ times gives an overall worst-case running time of $O(nk^3)$, which is much faster when k is small.